

Technical Specification Document (TSD)
Algorithmic Trading System

Document Control
Document Title: Technical Specification Document for Algorithmic Trading System
Version: 1.0
Date: April 24, 2025
Author: Senior Software Architect, Hedge Fund
Status: Draft
Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders
Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior Software Architect	Initial draft of TSD for Algorithmic Trading System

Approval

Role	Name	Signature	Date
Product Owner	[To be assigned]		
Head of Trading	[To be assigned]		
Compliance Officer	[To be assigned]		
Lead Developer	[To be assigned]		

Table of Contents

- 1. [Introduction](#)
 - 1.1. Purpose
 - 1.2. Scope
 - 1.3. Objectives
 - 1.4. Assumptions and Constraints
 - 1.5. Stakeholders
 - 1.6. Definitions and Acronyms
- 2. [System Overview](#)
 - 2.1. System Description
 - 2.2. System Goals
 - 2.3. Key Features
 - 2.4. System Context
- 3. [Technical Architecture](#)
 - 3.1. Architectural Overview
 - 3.2. Component Diagram
 - 3.3. Deployment Diagram
 - 3.4. Data Flow Diagram
 - 3.5. Technology Stack
- 4. [Detailed Component Specifications](#)
 - 4.1. Data Ingestion Service
 - 4.2. Data Processing Service
 - 4.3. Data Storage Service
 - 4.4. Strategy Engine
 - 4.5. Order Execution Module
 - 4.6. Risk Management Module
 - 4.7. AI Agents Framework
 - 4.8. Market Watch Dashboard
 - 4.9. Reporting Module
 - 4.10. Monitoring and Logging Service
- 5. [Data Specifications](#)
 - 5.1. Data Models
 - 5.2. Database Schema

- 5.3. Data Flow and Processing
- 5.4. Data Retention and Backup
- 6. [Interface Specifications](#)
 - 6.1. API Endpoints
 - 6.2. WebSocket Specifications
 - 6.3. User Interface Components
 - 6.4. External System Interfaces
- 7. [Non-Functional Specifications](#)
 - 7.1. Performance
 - 7.2. Scalability
 - 7.3. Security
 - 7.4. Reliability and Fault Tolerance
 - 7.5. Maintainability
 - 7.6. Usability
 - 7.7. Compliance
- 8. [Implementation Details](#)
 - 8.1. Development Environment
 - 8.2. Build and Deployment Process
 - 8.3. Testing Strategy
 - 8.4. Maintenance Plan
- 9. [Constraints and Dependencies](#)
 - 9.1. Hardware Constraints
 - 9.2. Software Constraints
 - 9.3. Regulatory Constraints
 - 9.4. Dependencies
- 10. [Missing Information and Best Practices](#)
 - 10.1. Missing Information
 - 10.2. Best Practices Applied
- 11. [Glossary](#)
- 12. [Appendices](#)
 - 12.1. Code Snippets
 - 12.2. Sequence Diagrams
 - 12.3. Mockups and Wireframes
 - 12.4. References

1. Introduction

1.1. Purpose

The Technical Specification Document (TSD) for the Algorithmic Trading System (ATS) provides a detailed technical blueprint for the development, deployment, and maintenance of a high-performance platform for simulated trading. This document specifies the system's architecture, components, interfaces, data models, and implementation details, serving as a comprehensive guide for the development team. The TSD translates system requirements into precise technical specifications, ensuring alignment among developers, architects, traders, and compliance officers. It facilitates the creation of a robust, scalable, and compliant system that meets the hedge fund's operational and strategic objectives.

1.2. Scope

The ATS is a sophisticated software platform designed for algorithmic paper trading across multiple asset classes, including stocks, indices, forex, and commodities, on major global exchanges such as NYSE, NASDAQ, LSE, and NSE. Integrated with the Interactive Brokers (IBKR) Paper Trading API, the system leverages Polars and DuckDB for data processing and storage, and incorporates AI agents for enhanced decision-making. The scope includes:

- **Trading Strategies:** Implementation of Trend-Following and Support and Resistance strategies for long and short positions.
- **Market Watch Dashboard:** A responsive web interface for real-time market monitoring, technical analysis, portfolio management, and simulated trading.
- **Data Management:** Efficient ingestion, processing, storage, and visualization of market data.
- **Order Execution:** Placement and monitoring of simulated trades via the IBKR API.

- **Risk Management:** Position sizing, stop-loss, take-profit, and portfolio-level risk controls.
- **AI Agents:** Modular components for market analysis, trade execution, and risk assessment.
- **Reporting and Analytics:** Trade performance metrics, backtesting results, portfolio analytics, and compliance reports.
- **Security and Compliance:** Encrypted communications, secure credential management, and adherence to financial regulations.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Support for cryptocurrencies or non-traditional assets.
- Mobile app development (dashboard will be mobile-responsive).
- Advanced AI model training (pre-trained models will be used).
- Premium third-party data sources for the initial phase.

1.3. Objectives

The ATS aims to achieve the following technical objectives:

1. **High Performance:** Achieve end-to-end latency of less than 100ms for data processing and trade execution.
2. **Scalability:** Support 10 concurrent users and 1TB of historical data, with a modular design for future enhancements.
3. **Reliability:** Ensure 99.9% uptime with robust fault-tolerant mechanisms.
4. **Security:** Implement encryption, authentication, and secure credential management to protect sensitive data.
5. **Maintainability:** Develop modular, well-documented components with at least 80% test coverage.
6. **Compliance:** Adhere to SEC, GDPR, MiFID II, and SEBI regulations through comprehensive audit trails and logging.
7. **Usability:** Deliver an intuitive dashboard requiring minimal training (less than 5 minutes).
8. **Extensibility:** Design the system to support future features such as live trading and alternative data integration.

1.4. Assumptions and Constraints

Assumptions:

- The IBKR Paper Trading API provides reliable, timely market data and order execution capabilities.
- Traders have intermediate knowledge of algorithmic trading and technical analysis.
- The Lenovo Legion Pro 5 (NVIDIA RTX 3060 GPU) meets performance requirements for development and deployment.
- Regulatory requirements for paper trading are less stringent than for live trading.
- The development team is proficient in Python, Polars, DuckDB, React, and FastAPI.
- Budget and resources are sufficient for a 3-6 month development cycle.

Constraints:

- Development timeline: 3-6 months to align with strategic goals.
- IBKR API rate limits: 50 requests/second for market data, 100 messages/second for orders.
- Budget: No premium third-party data sources or cloud hosting for initial deployment.
- Deployment: Local on Lenovo Legion Pro 5, no cloud infrastructure.
- Compliance: Must adhere to SEC, GDPR, MiFID II, and SEBI regulations.
- Hardware: Limited to existing Lenovo Legion Pro 5 infrastructure.

1.5. Stakeholders

Stakeholder	Role	Responsibilities
Head of Trading	Oversees trading operations	Defines strategy requirements, validates functionality
Traders	Primary end-users	Test system, provide usability feedback
Product Owner	Represents business interests	Prioritizes features, approves deliverables
Development Team	Builds and tests system	Implements specifications, ensures quality
Compliance Officer	Ensures regulatory adherence	Reviews system for compliance
IT Administrator	Manages infrastructure	Configures hardware, ensures uptime

1.6. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
 - **IBKR:** Interactive Brokers.
 - **OHLCV:** Open, High, Low, Close, Volume.
 - **VW SMA:** Volume-Weighted Simple Moving Average.
 - **MFI:** Money Flow Index.
 - **ATR:** Average True Range.
 - **Polars:** High-performance data processing library.
 - **DuckDB:** In-memory analytical database.
 - **FastAPI:** Python framework for building APIs.
 - **ib_insync:** Python library for IBKR API integration.
 - **Gann Fan:** Technical analysis tool for dynamic support/resistance.
 - **Fibonacci Retracement:** Percentage-based retracement levels.
-

2. System Overview

2.1. System Description

The Algorithmic Trading System (ATS) is a high-performance software platform designed for algorithmic paper trading, enabling simulated trades across stocks, indices, forex, and commodities on major global exchanges. Integrated with the IBKR Paper Trading API, the ATS leverages Polars for data processing, DuckDB for storage, and AI agents for enhanced decision-making. The system implements two positional trading strategies (Trend-Following and Support and Resistance), a responsive Market Watch Dashboard, and robust risk management protocols. Deployed locally on the Lenovo Legion Pro 5, the ATS employs a microservices-based, event-driven architecture to ensure performance, scalability, and maintainability.

2.2. System Goals

- **Performance:** Achieve <100ms end-to-end latency and process 10,000 data updates per second.
- **Reliability:** Ensure 99.9% uptime with fault-tolerant mechanisms.
- **Scalability:** Support 10 concurrent users and 1TB of historical data, with extensibility for future cloud deployment.
- **Security:** Implement end-to-end encryption and secure authentication.
- **Compliance:** Adhere to SEC, GDPR, MiFID II, and SEBI regulations.
- **Usability:** Provide an intuitive dashboard requiring minimal training.
- **Maintainability:** Develop modular components with >80% test coverage.

2.3. Key Features

- **Multi-Asset Trading:** Supports stocks (e.g., AAPL, RELIANCE), indices (e.g., S&P 500, NIFTY 50), forex (e.g., EUR/USD), and commodities (e.g., gold, crude oil).
- **Automated Strategies:** Executes Trend-Following and Support and Resistance strategies with dynamic position sizing.
- **Real-Time Monitoring:** Displays live prices, volume, and technical indicators via the Market Watch Dashboard.
- **Interactive Dashboard:** Offers candlestick charts, portfolio summaries, and simulated trading controls.
- **AI-Driven Insights:** Provides market analysis, trade execution, and risk assessment through modular AI agents.
- **Backtesting and Simulation:** Validates strategies against historical data in the IBKR paper trading environment.
- **Security:** Implements encrypted communications and secure credential management.
- **Reporting:** Generates trade performance metrics, portfolio analytics, and compliance reports.

2.4. System Context

The ATS interacts with the following entities:

- **External Systems:**
 - IBKR Paper Trading API: Provides market data and trade execution.
 - Future sources: News feeds, sentiment data (out of scope for initial phase).
- **Internal Systems:**
 - Portfolio management tools for benchmarking.
 - Compliance systems for transaction logging.
- **Users:**

- Traders: Interact via the Market Watch Dashboard.
 - Administrators: Manage configurations and monitor system health.
- **Environment:**
 - Deployed locally on Lenovo Legion Pro 5.
 - Dockerized services for consistency and isolation.

3. Technical Architecture

3.1. Architectural Overview

The ATS adopts a **microservices-based, event-driven architecture** to ensure modularity, low latency, and scalability. Key architectural principles include:

- **Separation of Concerns:** Isolates data management, trading logic, risk controls, and user interface.
- **Event-Driven Design:** Uses RabbitMQ for asynchronous communication to minimize latency.
- **Modularity:** Independent services enable maintainability and extensibility.
- **Security by Design:** Implements encryption and authentication at all layers.
- **Fault Tolerance:** Incorporates automatic reconnections and error handling for reliability.

The architecture is divided into:

- **Data Layer:** Handles ingestion, processing, storage, and caching of market data.
- **Business Logic Layer:** Manages trading strategies, order execution, risk controls, and AI agents.
- **Presentation Layer:** Provides the Market Watch Dashboard for user interaction.
- **Monitoring Layer:** Tracks system performance and logs activities.

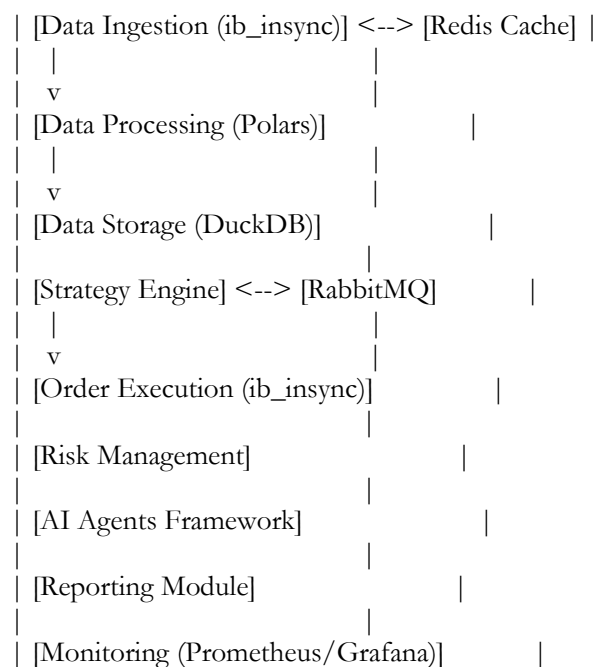
3.2. Component Diagram

[Market Watch Dashboard (React)]

|
| HTTP/WebSocket
v

[API Gateway (FastAPI)]

|
| REST/WebSocket
v



Components:

- **Data Ingestion:** Fetches real-time and historical data from IBKR API.
- **Data Processing:** Normalizes data and computes technical indicators.
- **Data Storage:** Stores time-series and transaction data in DuckDB.

- **Redis Cache:** Provides low-latency data access.
- **Strategy Engine:** Executes trading strategies and generates signals.
- **Order Execution:** Manages trade lifecycle via IBKR API.
- **Risk Management:** Enforces position sizing and risk controls.
- **AI Agents Framework:** Automates analysis, execution, and risk tasks.
- **Market Watch Dashboard:** Delivers a responsive user interface.
- **Reporting Module:** Generates performance and compliance reports.
- **Monitoring and Logging:** Tracks metrics and logs system activities.

3.3. Deployment Diagram

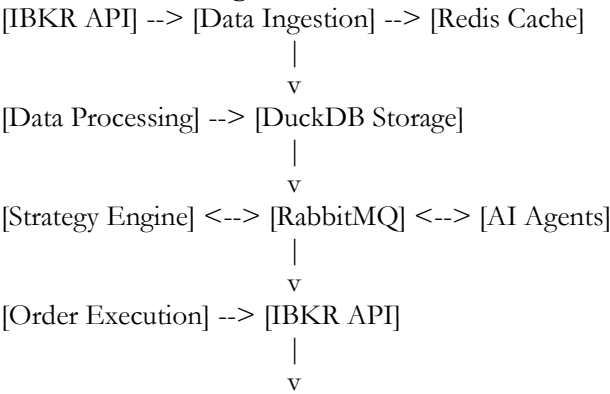


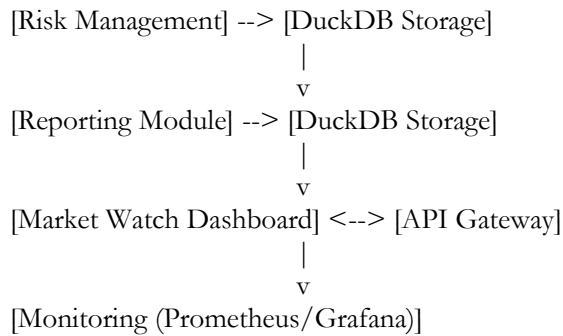
[IBKR Paper Trading API]

Deployment Details:

- **Host:** Single Lenovo Legion Pro 5 node with NVIDIA RTX 3060 GPU.
- **Containerization:** Docker containers for service isolation.
- **Reverse Proxy:** Nginx handles HTTP/WebSocket traffic and SSL termination.
- **Storage:** Local disk for DuckDB Parquet files.
- **Network:** Localhost for internal services, external connection to IBKR API.

3.4. Data Flow Diagram





Data Flow:

1. IBKR API streams OHLCV data to Data Ingestion.
2. Data is cached in Redis and forwarded to Data Processing.
3. Polars processes data, computes indicators, and stores results in DuckDB.
4. Strategy Engine consumes processed data, generates signals via RabbitMQ.
5. AI Agents analyze signals and trigger Order Execution.
6. Risk Management validates trades before execution.
7. Orders are sent to IBKR API and results stored in DuckDB.
8. Dashboard retrieves data via API Gateway for display.
9. Reporting Module queries DuckDB for reports.
10. Monitoring Service tracks metrics and logs.

3.5. Technology Stack

- **Backend:** Python 3.10, FastAPI for REST/WebSocket APIs, `ib_insync` for IBKR integration.
- **Data Processing:** Polars for high-performance data manipulation.
- **Database:** DuckDB with Parquet format for time-series storage.
- **Cache:** Redis for low-latency data access.
- **Messaging:** RabbitMQ for event-driven communication.
- **Frontend:** React 18, Tailwind CSS for styling, Plotly.js for charting.
- **AI/ML:** scikit-learn and TensorFlow for AI agent logic.
- **Containerization:** Docker and Docker Compose for service orchestration.
- **CI/CD:** GitHub Actions for automated build, test, and deployment.
- **Monitoring:** Prometheus for metrics collection, Grafana for visualization.
- **Logging:** Python logging with structured JSON output.
- **Security:** OAuth2 with JWT, TLS 1.3, AES-256 encryption.

4. Detailed Component Specifications

4.1. Data Ingestion Service

Purpose: Fetches real-time and historical OHLCV data from the IBKR Paper Trading API.

Specifications:

- **Input:** Ticker symbols, timeframes (e.g., 1m, 1d), API credentials (host: 127.0.0.1, port: 7497, client ID: 0, account: DUK221396).
- **Output:** OHLCV data in JSON format.
- **Process:**
 - Establish WebSocket connection using `ib_insync`.
 - Request real-time data with 5-10s polling interval.
 - Fetch historical data (up to 5 years) for backtesting.
 - Cache data in Redis with 60-second TTL.
 - Publish data to RabbitMQ for processing.
- **Rate Limits:** Adhere to 50 requests/second for market data.
- **Error Handling:**
 - Retry failed requests 3 times with exponential backoff (1s, 2s, 4s).
 - Log errors to JSON file with timestamp, service, and message.
 - Reconnect to IBKR API on disconnection.
- **Dependencies:** `ib_insync` (v0.9.70), Redis (v5.0), RabbitMQ (v3.12).
- **Performance:** Process 10,000 updates/second.

- **Security:** Encrypt API credentials using AES-256.

4.2. Data Processing Service

Purpose: Normalizes raw market data and computes technical indicators.

Specifications:

- **Input:** Raw OHLCV data from Redis.
- **Output:** Processed data with indicators (VW SMA, MFI, ATR).
- **Process:**
 - Normalize timestamps to UTC.
 - Clean data: Remove outliers ($>3\sigma$ from mean), interpolate missing values using linear method.
 - Compute indicators using Polars:
 - **VW SMA:**

$$\text{VW SMA} = \frac{\sum (\text{Price}_i \times \text{Volume}_i)}{\sum \text{Volume}_i}$$
 - **MFI:**

$$\text{MFI} = 100 - \frac{100}{1 + \frac{\text{Positive Money Flow}}{\text{Negative Money Flow}}}$$
 - **ATR:** VW SMA of True Range ($\max(\text{High} - \text{Low}, |\text{High} - \text{Previous Close}|, |\text{Low} - \text{Previous Close}|)$).
 - Store results in DuckDB.
 - Publish to RabbitMQ for Strategy Engine.
- **Error Handling:**
 - Skip corrupted records and log details.
 - Alert on persistent data quality issues.
- **Dependencies:** Polars (v0.20), Redis, RabbitMQ, DuckDB (v0.10).
- **Performance:** Compute indicators for 100 scrips in <1 second.
- **Security:** Restrict access to authorized services via OAuth2 tokens.

4.3. Data Storage Service

Purpose: Stores time-series data, trade logs, and portfolio data.

Specifications:

- **Input:** Processed OHLCV, trade logs, portfolio updates.
- **Output:** Queryable data in DuckDB.
- **Process:**
 - Store data in DuckDB using Parquet format.
 - Partition by date and ticker for query efficiency.
 - Retain data for 7 years to meet compliance requirements.
 - Perform daily backups to external storage.
- **Schema:** See Section 5.2.
- **Error Handling:**
 - Retry failed writes up to 3 times.
 - Log failures with record details.
- **Dependencies:** DuckDB, Parquet.
- **Performance:** Support 1TB historical data, $<100\text{ms}$ query latency.
- **Security:** Encrypt Parquet files with AES-256.

4.4. Strategy Engine

Purpose: Executes Trend-Following and Support and Resistance trading strategies.

Specifications:

- **Input:** Processed market data from RabbitMQ.
- **Output:** Trade signals (buy/sell) published to RabbitMQ.
- **Process:**
 - **Trend-Following Strategy:**

- **Entry Rules:**
 - Long: $\text{MFI}(14) > 34\text{-day VW SMA of MFI}(14)$ and $\text{price} > 55\text{-day VW SMA of Open}$.
 - Short: $\text{MFI}(14) < 34\text{-day VW SMA of MFI}(14)$ and $\text{price} < 55\text{-day VW SMA of Open}$.
- **Exit Rules:**
 - Long: Stop-loss at $\text{entry} - (2 \times 21\text{-day VW ATR})$, take-profit at $\text{entry} + (2 \times \text{ATR})$, or $\text{MFI}(14) < 21\text{-day VW SMA}$.
 - Short: Stop-loss at $\text{entry} + (2 \times \text{ATR})$, take-profit at $\text{entry} - (2 \times \text{ATR})$, or $\text{MFI}(14) > 21\text{-day VW SMA}$.
- **Position Sizing:**
 - Risk 1% of £10,000 account.
 - Formula:
$$\left[\text{Position Size} = \frac{\text{Account Balance} \times 0.01}{2 \times 21\text{-day VW ATR} \times \text{Lot Size}} \right]$$
- **Support and Resistance Strategy:**
 - **Entry Rules:**
 - Long: Price breaks above R1 or Fibonacci 61.8% with volume; or bounces off Gann Fan angle.
 - Short: Price breaks below S1 or Fibonacci 38.2% with volume; or rejected at Gann Fan angle.
 - **Exit Rules:**
 - Long: Take-profit at R2 or next Fibonacci, stop-loss below S1 or 1-2%.
 - Short: Take-profit at S2 or next Fibonacci, stop-loss above R1 or 1-2%.
 - **Position Sizing:**
 - Risk 1-2%.
 - Formula:
$$\left[\text{Position Size} = \frac{\text{Account Balance} \times \text{Risk \%}}{|\text{Entry Price} - \text{Stop Loss Price}|} \right]$$
- Publish signals to RabbitMQ with metadata (strategy, ticker, action).
- **Error Handling:**
 - Log invalid signals with data context.
 - Skip execution for corrupted inputs.
- **Dependencies:** Polars, RabbitMQ, DuckDB.
- **Performance:** Generate signals for 100 scrips in <1 second.
- **Security:** Restrict signal access to authorized services.

4.5. Order Execution Module

Purpose: Manages the trade lifecycle in the IBKR paper trading environment.

Specifications:

- **Input:** Trade signals from RabbitMQ.
- **Output:** Order status (pending, filled, canceled) stored in DuckDB.
- **Process:**
 - Validate signals: Check funds, price, and risk parameters.
 - Submit orders via `ib_insync`:
 - **Order Types:** Market, limit, stop, bracket (entry, stop-loss, take-profit).
 - Support partial fills.
 - Monitor order status via WebSocket.
 - Update portfolio and P&L in DuckDB.
 - Notify dashboard via WebSocket.
- **Rate Limits:** Adhere to 100 messages/second for orders.
- **Error Handling:**
 - Retry failed submissions 3 times with backoff.

- Log errors and alert critical failures.
- **Dependencies:** ib_insync, RabbitMQ, DuckDB.
- **Performance:** Execute 1,000 trades/minute.
- **Security:** Encrypt order data in transit and at rest.

4.6. Risk Management Module

Purpose: Enforces capital protection through risk controls.

Specifications:

- **Input:** Trade signals, portfolio data from DuckDB.
- **Output:** Validated signals or rejection alerts.
- **Process:**
 - **Position Sizing:**
 - Trend-Following: 1% risk, based on 2×21 -day VW ATR.
 - Support and Resistance: 1-2% risk, based on stop-loss distance.
 - **Stop-Loss and Take-Profit:**
 - Trend-Following: 2×21 -day VW ATR.
 - Support and Resistance: S1/R1 levels or 1-2%.
 - **Trailing Stop:**
 - Trend-Following: 1.5×21 -day VW ATR.
 - Support and Resistance: 2-3% or 2×14 -day ATR.
 - **Portfolio Risk:** Limit open risk to 5-10%.
 - **Drawdown Limits:**
 - 20% drawdown: Reduce risk to 0.5%.
 - 30% drawdown: Halt trading.
 - **Circuit Breakers:** Pause trading if price change >5% in 5 minutes.
 - **Currency Handling:** Convert to GBP using IBKR exchange rates, cache rates.
- **Error Handling:**
 - Reject non-compliant trades and log details.
 - Alert traders on circuit breaker activation.
- **Dependencies:** Polars, DuckDB, RabbitMQ.
- **Performance:** Validate trades in <10ms.
- **Security:** Restrict access to authorized services.

4.7. AI Agents Framework

Purpose: Automates trading tasks with human oversight.

Specifications:

- **Input:** Market data, trade signals, portfolio state from RabbitMQ/DuckDB.
- **Output:** Trade recommendations, risk adjustments published to RabbitMQ.
- **Agents:**
 - **Trade Execution Agent:**
 - Optimizes trade timing and price using reinforcement learning (Q-learning).
 - Inputs: Market volatility, order book depth.
 - **Risk Management Agent:**
 - Adjusts position sizing and exposure dynamically.
 - Uses rule-based logic and anomaly detection.
 - **Market Analysis Agent:**
 - Predicts trends using LSTM models.
 - Inputs: OHLCV, technical indicators.
 - **IBKR API Agent:**
 - Manages API interactions, optimizes call frequency.
 - Handles rate limits and errors.
- **Process:**
 - Consume data/signals via RabbitMQ.
 - Use scikit-learn for rule-based logic, TensorFlow for ML models.
 - Log decisions with SHAP explanations for transparency.
 - Require human approval for critical actions (e.g., large trades).
 - Allow pausing/overriding via dashboard.

- **Error Handling:**
 - Fallback to manual mode on agent failure.
 - Log errors with model inputs.
- **Dependencies:** scikit-learn (v1.3), TensorFlow (v2.15), RabbitMQ, DuckDB.
- **Performance:** Generate recommendations in <100ms.
- **Security:** Sandbox agents in Docker containers.

4.8. Market Watch Dashboard

Purpose: Provides a responsive web interface for market monitoring and trading.

Specifications:

- **Frontend:** React 18, Tailwind CSS for styling, Plotly.js for charting.
- **Backend:** FastAPI for REST and WebSocket APIs.
- **Components:**
 - **Script Selection:** Dropdown with auto-suggest, validates tickers via IBKR API.
 - **Real-Time Data Display:** Table showing bid, ask, last price, volume, % change.
 - **Interactive Charts:** Candlestick/line charts with zoom, pan, and indicators (VW SMA, MFI, ATR, RSI, MACD, Pivot Points, Fibonacci, Gann Fan).
 - **Portfolio Summary:** Table with positions, account value, unrealized P&L.
 - **Simulated Trading Panel:** Form for placing buy/sell orders (market, limit, stop, bracket).
 - **Watchlist:** List of saved scrips with volatility/volume highlights.
 - **Technical Analysis:** Displays market outlook and indicator status.
 - **News and Events:** Placeholder for future API integration, shows scrip-specific events.
- **Features:**
 - WebSocket updates every 5-10 seconds.
 - Responsive design for desktop and mobile (min-width: 320px).
 - Accessibility: WCAG 2.1 compliance (high contrast, keyboard navigation).
 - Export charts as PNG, save user settings in local storage.
- **Error Handling:**
 - Display user-friendly error messages.
 - Retry failed API calls 3 times.
- **Dependencies:** React, FastAPI, Plotly.js, Tailwind CSS.
- **Performance:** Render charts in <1 second, UI response <1 second.
- **Security:** Use OAuth2 with JWT for authentication, CSRF tokens for API calls.

4.9. Reporting Module

Purpose: Generates performance, analytics, and compliance reports.

Specifications:

- **Input:** Trade, portfolio, and market data from DuckDB.
- **Output:** Reports in CSV, PDF, and PNG formats.
- **Reports:**
 - **Trade Performance:** P&L, win rate, Sharpe ratio, max drawdown, equity curves.
 - **Portfolio Analytics:** Exposure, correlations, Value at Risk (VaR), expected shortfall.
 - **Backtesting Reports:** Historical performance, parameter sensitivity.
 - **Compliance Reports:** Transaction logs with timestamps, user IDs, immutable audit trails.
- **Process:**
 - Query DuckDB using SQL.
 - Aggregate data with pandas, visualize with Matplotlib.
 - Schedule reports via cron (daily, weekly).
 - Export via FastAPI endpoints.
- **Error Handling:**
 - Log query failures, retry exports up to 3 times.
 - Alert on report generation failures.
- **Dependencies:** pandas (v2.1), Matplotlib (v3.8), DuckDB, FastAPI.
- **Performance:** Generate reports for 1 year of data in <10 seconds.
- **Security:** Restrict report access to authorized users.

4.10. Monitoring and Logging Service

Purpose: Tracks system health, performance metrics, and logs activities.

Specifications:

- **Monitoring:**
 - **Metrics:** Latency, throughput, error rates, API response times.
 - **Tools:** Prometheus for metric collection, Grafana for visualization.
 - **Dashboards:** System health, trade performance, API usage.
 - **Alerts:** Notify via email/SMS on thresholds (e.g., latency >100ms).
 - **Logging:**
 - Structured JSON logs using Python logging.
 - Fields: timestamp, level, service, message, context.
 - Store logs in DuckDB for querying.
 - Retain logs for 7 years.
 - **Process:**
 - Services expose /metrics endpoint for Prometheus scraping (every 15s).
 - Grafana visualizes real-time and historical metrics.
 - Log all API calls, trade signals, and errors.
 - **Error Handling:**
 - Log monitoring failures, alert administrators.
 - Retry Prometheus scrapes on failure.
 - **Dependencies:** Prometheus (v2.45), Grafana (v10.0), DuckDB.
 - **Performance:** Collect metrics with <1ms overhead.
 - **Security:** Restrict monitoring access to admins via OAuth2.
-

5. Data Specifications

5.1. Data Models

OHLCV:

```
{
  "ticker": "string",
  "exchange": "string",
  "timestamp": "datetime",
  "open": "float",
  "high": "float",
  "low": "float",
  "close": "float",
  "volume": "int"
}
```

Trade Log:

```
{
  "trade_id": "string",
  "timestamp": "datetime",
  "ticker": "string",
  "strategy": "string",
  "action": "string",
  "quantity": "int",
  "price": "float",
  "status": "string"
}
```

Portfolio:

```
{
  "account_id": "string",
  "timestamp": "datetime",
  "ticker": "string",
  "quantity": "int",
  "entry_price": "float",
  "current_price": "float",
  "pnl": "float"
}
```

5.2. Database Schema

OHLCV Table:

```
CREATE TABLE ohlcv (  
    ticker VARCHAR,  
    exchange VARCHAR,  
    timestamp TIMESTAMP,  
    open DOUBLE,  
    high DOUBLE,  
    low DOUBLE,  
    close DOUBLE,  
    volume BIGINT,  
    PRIMARY KEY (ticker, timestamp)  
);
```

Trade Log Table:

```
CREATE TABLE trade_log (  
    trade_id VARCHAR PRIMARY KEY,  
    timestamp TIMESTAMP,  
    ticker VARCHAR,  
    strategy VARCHAR,  
    action VARCHAR,  
    quantity BIGINT,  
    price DOUBLE,  
    status VARCHAR  
);
```

Portfolio Table:

```
CREATE TABLE portfolio (  
    account_id VARCHAR,  
    timestamp TIMESTAMP,  
    ticker VARCHAR,  
    quantity BIGINT,  
    entry_price DOUBLE,  
    current_price DOUBLE,  
    pnl DOUBLE,  
    PRIMARY KEY (account_id, ticker, timestamp)  
);
```

Partitioning: By date and ticker to optimize query performance.

5.3. Data Flow and Processing

- **Ingestion:** IBKR API streams OHLCV data to Data Ingestion, cached in Redis.
- **Processing:** Polars normalizes data, computes indicators, and stores results in DuckDB.
- **Access:** Dashboard and Reporting Module query DuckDB via FastAPI.
- **Cache:** Redis stores recent OHLCV data with 60-second TTL.
- **Flow:**
 - Raw data → Redis → Polars → DuckDB.
 - Signals → RabbitMQ → Order Execution → DuckDB.
 - Portfolio updates → DuckDB → Dashboard.

5.4. Data Retention and Backup

- **Retention:** Store data for 7 years to meet compliance requirements.
- **Backup:**
 - Daily cron job exports Parquet files to external SSD.
 - Compression: Use Parquet's native compression.
 - Restore Time Objective (RTO): <1 hour.
- **Security:** Encrypt backup files with AES-256.

6. Interface Specifications

6.1. API Endpoints

GET /market-data:

- **Parameters:** ticker (string), timeframe (string, e.g., "1m", "1d").
- **Response:**
- [
- {
- "ticker": "AAPL",
- "exchange": "NASDAQ",
- "timestamp": "2025-04-24T10:00:00Z",
- "open": 150.25,
- "high": 151.10,
- "low": 149.80,
- "close": 150.90,
- "volume": 1000000
- }
-]
- **Authentication:** OAuth2 JWT.
- **Rate Limit:** 100 requests/minute.

POST /orders:

- **Body:**
- {
- "ticker": "AAPL",
- "action": "BUY",
- "quantity": 100,
- "price": 150.90,
- "type": "LIMIT"
- }
- **Response:**
- {
- "order_id": "O12345",
- "status": "PENDING"
- }
- **Authentication:** OAuth2 JWT.
- **Rate Limit:** 50 requests/minute.

GET /portfolio:

- **Parameters:** account_id (string).
- **Response:**
- [
- {
- "account_id": "DUK221396",
- "ticker": "AAPL",
- "quantity": 100,
- "entry_price": 150.90,
- "current_price": 151.50,
- "pnl": 60.00
- }
-]
- **Authentication:** OAuth2 JWT.

6.2. WebSocket Specifications

Endpoint: /ws/market-data

- **Message Format:** Same as GET /market-data response.
- **Frequency:** Updates every 5-10 seconds.
- **Authentication:** JWT token in connection header.
- **Error Handling:** Reconnect on disconnection, log failures.

6.3. User Interface Components

- **Layout:** Grid-based with:

- Top: Scrip selector, watchlist.
- Left: Real-time data table, technical analysis panel.
- Center: Candlestick chart with indicators.
- Right: Portfolio summary, trading panel.
- Bottom: News/events section.
- **Components:**
 - **Scrip Selector:** React dropdown with auto-suggest, fetches tickers via API.
 - **Data Table:** Displays bid, ask, volume, % change; updates via WebSocket.
 - **Chart:** Plotly.js candlestick with zoom, pan, and indicator overlays.
 - **Portfolio:** Table with filters for asset, exchange, strategy.
 - **Trading Panel:** Form with inputs for ticker, quantity, price, order type.
- **Features:**
 - Responsive: Supports 320px to 1920px widths.
 - Accessibility: WCAG 2.1 (ARIA labels, keyboard navigation).
 - Tooltips: Explain indicators and actions.

6.4. External System Interfaces

- **IBKR Paper Trading API:**
 - **Connection:** Host: 127.0.0.1, Port: 7497, Client ID: 0, Account: DUK221396.
 - **Methods:**
 - reqMktData: Fetch real-time OHLCV.
 - placeOrder: Submit market/limit/stop/bracket orders.
 - reqHistoricalData: Fetch historical data.
 - **Rate Limits:** 50 req/s (data), 100 msg/s (orders).
 - **Security:** Encrypt credentials, use TLS 1.3.
- **Future Interfaces:** Placeholder for REST/WebSocket news APIs (e.g., Reuters, Bloomberg).

7. Non-Functional Specifications

7.1. Performance

- **Latency:** <100ms end-to-end (data ingestion to trade execution).
- **Throughput:** 10,000 data updates/second, 1,000 trades/minute.
- **Indicator Calculation:** <1 second for 100 scrips.
- **Dashboard Refresh:** Every 5-10 seconds.
- **Backtesting:** Process 5 years of data in <10 minutes.
- **Implementation:**
 - Use Polars for parallel processing.
 - Cache data in Redis for low-latency access.
 - Optimize WebSocket for real-time updates.

7.2. Scalability

- **Vertical:** Optimize for Lenovo Legion Pro 5 GPU/CPU.
- **Horizontal:** Microservices architecture for future cloud deployment.
- **Data:** Handle 1TB historical data.
- **Users:** Support 10 concurrent dashboard users.
- **Implementation:**
 - Stateless services for load balancing.
 - RabbitMQ for distributed event handling.
 - DuckDB partitioning for query scalability.

7.3. Security

- **Authentication:** OAuth2 with JWT, multi-factor authentication (MFA).
- **Data Protection:** TLS 1.3 for communications, AES-256 for stored data.
- **Credentials:** Store in encrypted vault (e.g., HashiCorp Vault).
- **API Security:** Rate limiting, CSRF tokens.
- **AI Agents:** Run in sandboxed Docker containers.
- **Implementation:**
 - Nginx for SSL termination.
 - Keycloak for OAuth2/MFA.

- Regular OWASP ZAP security scans.

7.4. Reliability and Fault Tolerance

- **Uptime:** 99.9%.
- **Error Handling:** Auto-reconnect to IBKR API on disconnection.
- **Data Integrity:** Validate market data for consistency.
- **Circuit Breakers:** Halt trading on anomalies (e.g., >5% price change in 5 minutes).
- **Backup:** Daily backups with <1 hour RTO.
- **Implementation:**
 - Retry logic in `ib_insync` for API calls.
 - Health checks in Docker Compose.
 - Prometheus alerts for anomalies.

7.5. Maintainability

- **Code Coverage:** >80% with unit and integration tests.
- **Modularity:** Independent microservices.
- **Logging:** Structured JSON logs for querying.
- **Documentation:** OpenAPI specs, Confluence guides.
- **Implementation:**
 - Follow clean code principles.
 - Use GitHub for version control.
 - Automate documentation with OpenAPI.

7.6. Usability

- **Navigation:** Learn dashboard in <5 minutes.
- **UI Response:** <1 second for interactions.
- **Accessibility:** WCAG 2.1 compliance.
- **Implementation:**
 - Tailwind CSS for consistent styling.
 - User testing with traders.
 - Tooltips and help modals for clarity.

7.7. Compliance

- **Regulations:** SEC, GDPR, MiFID II, SEBI.
- **Logging:** Immutable trade and order logs.
- **Retention:** Store logs for 7 years.
- **Monitoring:** Detect market abuse (e.g., spoofing).
- **Implementation:**
 - DuckDB for immutable audit trails.
 - Compliance review at each sprint.
 - Log all user actions with timestamps.

8. Implementation Details

8.1. Development Environment

- **OS:** Ubuntu 22.04 on Lenovo Legion Pro 5.
- **IDE:** VS Code with Python, React extensions.
- **Version Control:** GitHub with branch protection.
- **Dependencies:** Managed via Poetry (Python), npm (React).
- **Containerization:** Docker Compose for local development.

8.2. Build and Deployment Process

- **Build:**
 - Python: Compile with Poetry, create Docker images.
 - React: Build with npm, bundle with Webpack.
- **CI/CD:** GitHub Actions pipeline:
 - **Triggers:** Push to main, pull requests.
 - **Steps:** Lint, test, build, push images to local registry.
 - **Tests:** Unit (pytest), integration, performance (Locust).
- **Deployment:**
 - Deploy to Lenovo Legion Pro 5 via Docker Compose.

- Manual approval for production.
 - Rollback: Revert to previous image on failure.
 - **Configuration:** Environment variables in .env files.
- ### 8.3. Testing Strategy
- **Unit Tests:** Test individual components with pytest (e.g., indicator calculations).
 - **Integration Tests:** Validate interactions (e.g., Strategy Engine to Order Execution).
 - **System Tests:** Test end-to-end workflows (e.g., signal to trade execution).
 - **Performance Tests:** Measure latency/throughput with Locust.
 - **Security Tests:** OWASP ZAP for vulnerabilities.
 - **Backtesting Tests:** Validate strategies against historical data.
 - **Coverage:** Achieve >80% with coverage.py.
 - **Tools:** pytest, Locust, OWASP ZAP.
- ### 8.4. Maintenance Plan
- **Updates:** Monthly patches, quarterly feature releases.
 - **Bug Fixes:** Prioritize critical issues (<24 hours).
 - **Monitoring:** Daily health checks via Grafana.
 - **Documentation:** Update Confluence with changes.
 - **Support:** On-call rotation for incidents.
 - **Deprecation:** Phase out outdated libraries within 6 months.
-

9. Constraints and Dependencies

9.1. Hardware Constraints

- **Host:** Lenovo Legion Pro 5 with NVIDIA RTX 3060 GPU.
- **Deployment:** Local only, no cloud infrastructure.
- **Optimization:** Leverage GPU for Polars/TensorFlow, CPU for DuckDB.

9.2. Software Constraints

- **Languages:** Python 3.10, JavaScript (React 18).
- **Data Sources:** Limited to IBKR API, no premium sources.
- **Containerization:** Docker for service isolation.

9.3. Regulatory Constraints

- **Regulations:** Comply with SEC, GDPR, MiFID II, SEBI.
- **Logging:** Immutable logs for 7 years.
- **Monitoring:** Detect and report market abuse.

9.4. Dependencies

- **External:** IBKR Paper Trading API availability.
 - **Libraries:** Polars, DuckDB, ib_insync, scikit-learn, TensorFlow.
 - **Hardware:** Lenovo Legion Pro 5.
 - **Team:** Expertise in Python, React, AI/ML.
-

10. Missing Information and Best Practices

10.1. Missing Information

- Exact formulas for volume-weighted indicators.
- Detailed trader personas and workflows.
- Specific regulatory requirements for paper trading.
- Performance benchmarks for Lenovo Legion Pro 5.
- News feed providers for future integration.

10.2. Best Practices Applied

- **Indicator Formulas:** Adopted standard VW calculations (e.g., $VW\ SMA = \frac{\sum(Price \times Volume)}{\sum Volume}$). **Reason:** Industry-standard, ensures accuracy and compatibility.
- **Personas:** Assumed intermediate traders with technical analysis knowledge. **Reason:** Typical for hedge funds, focuses usability on core users.
- **Compliance:** Implemented immutable audit trails and 7-year retention. **Reason:** Aligns with common financial regulations, prepares for live trading.

- **Performance:** Targeted <100ms latency based on high-frequency trading standards. **Reason:** Ensures competitiveness in fast-paced markets.
- **News Integration:** Planned placeholders for Reuters/Bloomberg APIs. **Reason:** Reliable sources for financial data, extensible for future phases.

11. Glossary

- **ATS:** Algorithmic Trading System.
 - **IBKR:** Interactive Brokers.
 - **OHLCV:** Open, High, Low, Close, Volume.
 - **VW SMA:** Volume-Weighted Simple Moving Average.
 - **MFI:** Money Flow Index.
 - **ATR:** Average True Range.
 - **Polars:** High-performance data processing library.
 - **DuckDB:** In-memory analytical database.
-

12. Appendices

12.1. Code Snippets

Data Ingestion (Python):

```
from ib_insync import IB, MarketOrder
import redis
import json
```

```
class DataIngestion:
```

```
    def __init__(self):
        self.ib = IB()
        self.redis = redis.Redis(host='localhost', port=6379)
        self.ib.connect('127.0.0.1', 7497, clientId=0)
```

```
    async def fetch_data(self, ticker, timeframe):
        contract = self.ib.qualifyContracts(ticker)[0]
        data = self.ib.reqMktData(contract, "", False, False)
        self.redis.setex(f'{ticker}:{timeframe}', 60, json.dumps(data))
        return data
```

React Component (Dashboard):

```
import React, { useEffect } from 'react';
import Plotly from 'plotly.js';
```

```
const Chart = ({ ticker }) => {
    useEffect(() => {
        fetch(`/market-data?ticker=${ticker}`)
            .then(res => res.json())
            .then(data => {
                Plotly.newPlot('chart', [{
                    x: data.map(d => d.timestamp),
                    open: data.map(d => d.open),
                    high: data.map(d => d.high),
                    low: data.map(d => d.low),
                    close: data.map(d => d.close),
                    type: 'candlestick'
                }]);
            });
    }, [ticker]);
```

```
    return <div id="chart"></div>;
};
```

12.2. Sequence Diagrams

Trade Execution:

Trader -> Dashboard: Select ticker, submit order

Dashboard -> API Gateway: POST /orders

API Gateway -> Strategy Engine: Validate signal

Strategy Engine -> Risk Management: Check risk

Risk Management -> Order Execution: Place order

Order Execution -> IBKR API: Submit order

IBKR API -> Order Execution: Order status

Order Execution -> Dashboard: Update status

12.3. Mockups and Wireframes

- **Dashboard:** Grid layout with scrip selector, real-time data, candlestick chart, portfolio, and trading panel.
- **Chart:** Candlestick with overlay indicators, zoom controls, and export button.
- **Trading Panel:** Form with inputs for ticker, quantity, price, and order type.

12.4. References

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
- Polars: <https://pola.rs/>
- DuckDB: <https://duckdb.org/>
- FastAPI: <https://fastapi.tiangolo.com/>
- React: <https://reactjs.org/>
- SEC Regulations: <https://www.sec.gov/>